

ECOLE SUPERIEURE PRIVEE DES TECHNOLOGIES DE L'INFORMATION
ET DE MANAGEMENT DE NABEUL



Rapport de Mini-Projet

Plateforme d'Analyse des logs de Sécurité et Authentification

3^{ème} année cycle ingénieur

Elaboré par :

Imen HADJ TAIEB (GL)

Oussema OUERGHI (SR)

Encadrante universitaire : **Mme AYED Dalel**

Année universitaire
2025-2026

Dédicaces

À nos parents

Pour tout le soutien qu'ils ont apporté durant nos études et les nombreux sacrifices qu'ils ont consentis pour nous amener à ce niveau. Nous espérons qu'un jour nous pourrons leur rendre une partie de ce qu'ils ont fait pour nous, que Dieu les bénisse avec bonheur et longue vie.

À nos frères et sœurs

Pour leur soutien moral et pour leur amour. Nous vous souhaitons sincèrement une vie pleine de joie et de succès.

Remerciements

Au terme de ce travail,

Nous souhaitons exprimer nos sincères remerciements et reconnaissances. Tout d'abord, nous sommes reconnaissants envers notre DIEU pour nous avoir accordé le courage, la force et la détermination nécessaires pour mener à bien ce modeste travail.

Nous tenons également à exprimer notre profonde gratitude envers tous ceux qui ont apporté leur soutien et ont contribué de manière significative à la réalisation de ce projet.

Nos familles méritent une mention spéciale pour leur soutien constant tout au long de cette aventure.

Un remerciement particulier est adressé à Madame AYED Dalel pour ses conseils éclairés, qui ont grandement facilité le processus de collecte d'informations et ont contribué à la réussite de notre mini-projet. Enfin, nous souhaitons témoigner notre reconnaissance à toute l'équipe de notre chère institution, l'ITBS, qui a joué un rôle crucial dans notre formation. Leur engagement envers notre apprentissage a été inestimable, et nous sommes reconnaissants pour l'environnement d'apprentissage enrichissant qu'ils ont créé.

Ces remerciements reflètent notre profonde appréciation envers ceux qui ont été des sources d'inspiration, de guidance et de soutien tout au long de ce parcours. Nous sommes reconnaissants de faire partie d'une communauté aussi solidaire et bienveillante.

Table des matières

Introduction générale	1
1 Présentation du sujet d'étude	2
1.1 Introduction	2
1.2 Contexte	2
1.2.1 Problématique	2
1.3 Solution Proposée	3
2 Notion de Big Data Framework	4
2.1 Introduction	4
2.1.1 Définition et Caractéristiques	4
2.1.2 Importance dans le Contexte de la Sécurité	4
2.2 Architecture Big Data pour la Sécurité	5
2.2.1 Stack ELK comme Solution Big Data	5
2.2.2 Caractéristiques Big Data d'Elasticsearch	5
2.3 Les Trois Couches du Framework Big Data	5
2.3.1 Couche d'Ingestion des Données	5
2.3.2 Couche de Stockage des Données	6
2.3.3 Couche de Traitement des Données	6
2.4 Conclusion	6
3 Architecture technique	7
3.1 Architecture Technique	7
3.1.1 Stack Technologique	7
3.1.2 Schéma d'Architecture	8
3.2 Environnement de Développement	8
3.2.1 Environnement Matériel	8
3.2.2 Configuration Logicielle	9
3.3 Implémentation des Composants	9
3.3.1 Docker Compose Principal	9
3.3.2 Application Flask	9
3.3.3 Pipeline Logstash	10
3.3.4 Génération de Logs de Test	10
3.4 Configuration et Déploiement	11
3.4.1 Démarrage de la Plateforme	11
3.4.2 Accès aux Services	11

4	Réalisation	12
4.1	Introduction	12
4.2	Conception des Tableaux de Bord de Sécurité	12
4.3	Dashboard d'Analyse Avancée	12
4.4	Implémentation des Visualisations dans Kibana	14
4.4.1	Configuration des Index Patterns	14
4.4.2	Création des Visualisations	14
4.4.2.1	Création du Dashboard Principal	15
4.5	Intégration et Tests	15
4.5.1	Tableaux de Bord Kibana	15
4.5.2	Tests de l'API avec Postman	16
4.5.3	Utilisation de Kibana Canvas	16
4.6	Tests d'Intégration avec Postman	17
4.6.1	Configuration des Requêtes API	17
4.7	Optimisations et Bonnes Pratiques	17
4.7.1	Optimisation des Performances	17
4.8	Conclusion	17
	Conclusion générale	17
	Webographie	18

Table des figures

2.1	Architecture Big Data basée sur la stack ELK	5
3.1	Architecture globale de la plateforme	8
3.2	Interface principale de l'application Flask	10
4.1	Interface de recherche avancée avec filtres	13
4.2	Création de l'Index Pattern dans Kibana	14
4.3	Création des visualisations Kibana	14
4.4	Tableau de bord principal de monitoring de sécurité	15
4.5	Rapport Canvas avec visualisations intégrées	16
4.6	Test d'ingestion de logs via Postman	17

Liste des tableaux

2.1	Caractéristiques Big Data d'Elasticsearch	5
2.2	Stratégies de stockage Big Data	6
3.1	Configuration logicielle	9
3.2	Accès aux services	11

Introduction générale

Avec la transformation numérique accélérée des entreprises, les systèmes d'information génèrent quotidiennement un volume considérable de données, en particulier sous forme de logs issus des applications, des équipements réseau et des mécanismes de sécurité. Ces journaux constituent une source essentielle d'information pour la détection des incidents, l'analyse des comportements anormaux et le renforcement de la sécurité globale des infrastructures informatiques. Toutefois, la gestion manuelle de ces données devient rapidement inefficace face à leur volume, leur diversité et leur vitesse de génération.

C'est dans ce cadre que s'inscrit ce projet, dont l'objectif est la conception et la réalisation d'une plateforme de monitoring et d'analyse des logs de sécurité basée sur un framework Big Data. La solution proposée repose sur la stack ELK (Elasticsearch, Logstash et Kibana), reconnue pour ses performances en matière d'indexation, de recherche et de visualisation des données. Elle est complétée par une application backend développée avec Flask, permettant l'interaction avec les données via des API REST, ainsi que par une architecture conteneurisée assurant la portabilité et la facilité de déploiement.

Ce travail vise à répondre aux problématiques liées à la dispersion des logs, à la complexité de leur analyse et au manque de visibilité sur les événements de sécurité. Il met en œuvre une approche structurée intégrant les concepts du Big Data, de la sécurité des systèmes d'information et de l'ingénierie logicielle moderne, tout en proposant une solution pratique et exploitable.

Le présent rapport est organisé comme suit :

- **Chapitre 1 : Présentation du sujet d'étude**

Ce chapitre introduit le contexte général du projet, expose la problématique liée à la gestion des logs de sécurité et présente la solution proposée pour y remédier.

- **Chapitre 2 : Notion de Big Data Framework**

Il présente les concepts fondamentaux du Big Data, leur importance dans le domaine de la sécurité, ainsi que l'architecture Big Data adoptée, en mettant l'accent sur la stack ELK et ses différentes couches fonctionnelles.

- **Chapitre 3 : Architecture technique**

Ce chapitre détaille l'architecture globale de la plateforme, les choix technologiques effectués, l'environnement de développement, l'implémentation des composants ainsi que les étapes de configuration et de déploiement.

- **Chapitre 4 : Réalisation**

Il est consacré à la mise en œuvre concrète de la solution, incluant la conception des tableaux de bord de sécurité, l'intégration des composants, les tests réalisés et les optimisations appliquées.

Chapitre 1

Présentation du sujet d'étude

1.1 Introduction

Ce chapitre présente le contexte général du projet, la problématique rencontrée ainsi que la solution proposée afin de répondre efficacement aux besoins de supervision et d'analyse des événements de sécurité.

1.2 Contexte

Dans le cadre de la sécurisation des systèmes informatiques de l'entreprise, une quantité importante de logs de sécurité est générée quotidiennement. Ces logs proviennent de sources hétérogènes telles que les pare-feu, les systèmes de détection d'intrusion (IDS), les mécanismes d'authentification, ainsi que les accès aux ressources sensibles.

La diversité et le volume élevé de ces données rendent leur exploitation difficile sans un outil dédié. Une analyse efficace de ces logs est pourtant indispensable pour détecter les tentatives d'intrusion, anticiper les incidents de sécurité et garantir la conformité aux normes et politiques de sécurité en vigueur.

1.2.1 Problématique

L'analyse de la situation actuelle met en évidence plusieurs limitations majeures :

- **Dispersion des logs** : Les journaux sont stockés sur plusieurs serveurs et équipements, sans mécanisme de centralisation, ce qui complique leur exploitation globale.
- **Diagnostic complexe** : L'analyse manuelle des logs est chronophage, sujette aux erreurs humaines et peu adaptée à des volumes de données importants.
- **Visibilité limitée** : L'absence d'outils analytiques avancés rend difficile l'identification des schémas d'attaques, des anomalies et des incidents critiques.
- **Interface unifiée manquante** : Le centre opérationnel de sécurité (SOC) ne dispose pas de tableaux de bord centralisés permettant une supervision en temps réel.

Ces contraintes limitent la capacité de l'entreprise à réagir rapidement aux incidents de sécurité et à prendre des décisions basées sur des données fiables.

1.3 Solution Proposée

Afin de répondre aux problématiques identifiées, ce projet vise à concevoir, développer et déployer une plateforme complète de monitoring et d'analyse des logs de sécurité.

La solution proposée permet de :

- Centraliser la collecte des logs d'authentification et de sécurité provenant de différentes sources.
- Indexer les données afin de permettre des recherches rapides et efficaces des incidents.
- Visualiser les menaces, anomalies et événements critiques à travers des tableaux de bord interactifs.
- Générer des rapports facilitant le suivi des incidents et la conformité aux exigences de sécurité.

Cette plateforme offre ainsi une vision globale et unifiée de l'état de sécurité du système d'information, tout en améliorant la capacité de détection et de réaction face aux incidents.

Conclusion

Dans ce chapitre, nous avons présenté le sujet d'étude en définissant le contexte général du projet et les enjeux liés à la gestion des logs de sécurité. Les chapitres suivants seront consacrés à la conception et à la réalisation technique de cette solution.

Chapitre 2

Notion de Big Data Framework

2.1 Introduction

Ce chapitre présente les notions fondamentales du Big Data, met en évidence son importance dans le domaine de la sécurité informatique et décrit l'architecture Big Data adoptée dans le cadre de notre plateforme de monitoring des logs de sécurité.

2.1.1 Définition et Caractéristiques

Le Big Data désigne des ensembles de données dont la taille, la complexité et la vitesse de génération nécessitent des outils et des architectures spécifiques pour leur exploitation. Dans le cadre de notre plateforme de monitoring de logs de sécurité, les données traitées répondent pleinement aux **5V du Big Data** :

1. **Volume** : Génération de milliers de logs par seconde provenant de multiples sources.
2. **Vélocité** : Flux de données continus nécessitant un traitement quasi temps réel.
3. **Variété** : Multiplicité des formats de logs (CSV, JSON, Syslog, Apache logs, etc.).
4. **Véracité** : Nécessité de garantir l'intégrité, la fiabilité et la cohérence des données de sécurité.
5. **Valeur** : Capacité à extraire des informations pertinentes pour la détection des menaces et la prise de décision.

2.1.2 Importance dans le Contexte de la Sécurité

Les logs de sécurité modernes présentent des caractéristiques Big Data qui rendent les solutions classiques inadaptées. En effet, les systèmes traditionnels de gestion de bases de données relationnelles (RDBMS) ne sont pas conçus pour :

- le traitement en temps réel de flux massifs de données,
- l'analyse de grandes quantités de données historiques,
- la corrélation d'événements issus de sources hétérogènes,
- la détection d'anomalies et de comportements suspects à grande échelle.

L'adoption d'un framework Big Data permet ainsi d'améliorer la visibilité globale, la rapidité de détection des incidents et la capacité de réaction face aux menaces de sécurité.

2.2 Architecture Big Data pour la Sécurité

2.2.1 Stack ELK comme Solution Big Data

La stack ELK (Elasticsearch, Logstash, Kibana) constitue un framework Big Data spécialisé dans la gestion, l’indexation et l’analyse des logs. Son architecture distribuée répond efficacement aux contraintes de volume, de vélocité et de variété des données de sécurité.

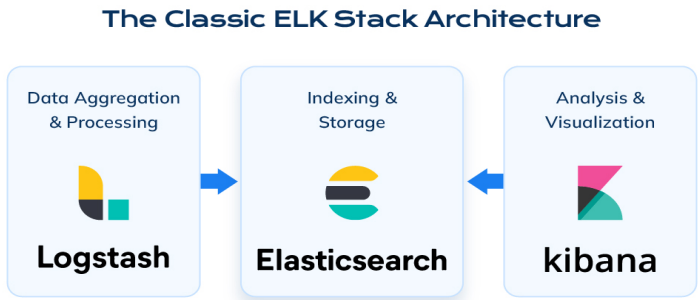


FIGURE 2.1 – Architecture Big Data basée sur la stack ELK

Cette architecture permet de collecter les logs via Logstash, de les indexer dans Elasticsearch et de les analyser visuellement à l’aide de Kibana.

2.2.2 Caractéristiques Big Data d’Elasticsearch

Elasticsearch constitue le cœur de la plateforme Big Data de monitoring. Ses principales caractéristiques sont présentées dans le tableau suivant :

Caractéristique	Avantage Big Data
Distribution horizontale	Scalabilité linéaire par ajout de nœuds
Sharding automatique	Répartition transparente des données
Réplication	Haute disponibilité et tolérance aux pannes
Index inversé	Recherche full-text très performante
Agrégations temps réel	Analyse instantanée sur de grands volumes
API RESTful	Intégration simple avec d’autres systèmes

TABLE 2.1 – Caractéristiques Big Data d’Elasticsearch

2.3 Les Trois Couches du Framework Big Data

2.3.1 Couche d’Ingestion des Données

La couche d’ingestion assure la collecte et le prétraitement des données provenant de différentes sources. Les outils utilisés sont :

- **Logstash** : Filtrage, transformation et enrichissement des logs.
- **Filebeat** : Collecte légère des logs sur les hôtes.

- **Kafka** : Système de messagerie distribué permettant d'absorber les pics de charge.
- **Ingest Nodes** : Prétraitement natif au niveau d'Elasticsearch.

2.3.2 Couche de Stockage des Données

Le stockage Big Data repose sur une gestion hiérarchisée des données en fonction de leur ancienneté et de leur fréquence d'accès :

Type	Technologie	Usage dans la plateforme
Hot Storage	SSD / NVMe	Données récentes (moins de 7 jours)
Warm Storage	HDD RAID	Données intermédiaires (7 à 30 jours)
Cold Storage	Object Storage	Archivage à long terme (plus de 30 jours)
Index Lifecycle Management	ILM Policy	Gestion automatique du cycle de vie des index
Compression	LZ4 / DEFLATE	Optimisation de l'espace de stockage

TABLE 2.2 – Stratégies de stockage Big Data

2.3.3 Couche de Traitement des Données

Le traitement des données Big Data s'effectue selon trois modes complémentaires :

1. Traitement par lot (Batch) :

- Analyse historique des logs
- Génération de rapports périodiques
- Nettoyage et archivage des données

2. Traitement en flux (Stream) :

- Détection d'anomalies en temps réel
- Génération d'alertes immédiates
- Suivi du comportement des utilisateurs

3. Traitement interactif :

- Recherche ad-hoc via Kibana
- Investigation des incidents de sécurité
- Analyse approfondie par les analystes SOC

2.4 Conclusion

Ce chapitre a présenté les notions fondamentales des frameworks Big Data et leur importance dans le domaine de la sécurité informatique. Nous avons mis en évidence les caractéristiques Big Data des logs de sécurité et justifié le recours à une architecture distribuée basée sur la stack ELK. Enfin, l'étude des différentes couches du framework Big Data a permis de démontrer comment la collecte, le stockage et le traitement des données sont organisés afin de répondre aux exigences de performance, de scalabilité et de fiabilité. Ces concepts constituent une base essentielle pour comprendre l'architecture et les choix techniques adoptés dans la réalisation de notre plateforme de monitoring.

Chapitre 3

Architecture technique

Introduction

Ce chapitre présente en détail l'architecture technique adoptée, les choix technologiques effectués ainsi que l'environnement de développement utilisé. Nous décrivons également l'implémentation des différents composants du système, le processus de déploiement et les tests réalisés afin de valider le bon fonctionnement de la plateforme.

3.1 Architecture Technique

3.1.1 Stack Technologique

La plateforme repose sur une architecture moderne orientée microservices, entièrement conteneurisée afin d'assurer la portabilité, la scalabilité et la facilité de déploiement. Les technologies suivantes ont été sélectionnées pour répondre aux exigences fonctionnelles et non fonctionnelles du projet :

- **Backend** : Python avec le framework Flask pour le développement d'une API REST légère et performante.
- **Base de données** :
 - **Elasticsearch** pour l'indexation, la recherche rapide et l'analyse des logs de sécurité.
 - **MongoDB** pour le stockage des métadonnées, des configurations et des informations applicatives.
- **Ingestion des logs** : Logstash pour la collecte, le filtrage et la transformation des logs avant leur indexation.
- **Visualisation** : Kibana pour la création de tableaux de bord interactifs et l'analyse visuelle des événements.
- **Conteneurisation** : Docker et Docker Compose pour l'orchestration des différents services.

3.1.2 Schéma d'Architecture

La Figure ci-dessous illustre l'architecture globale de la plateforme et les interactions entre ses différents composants.

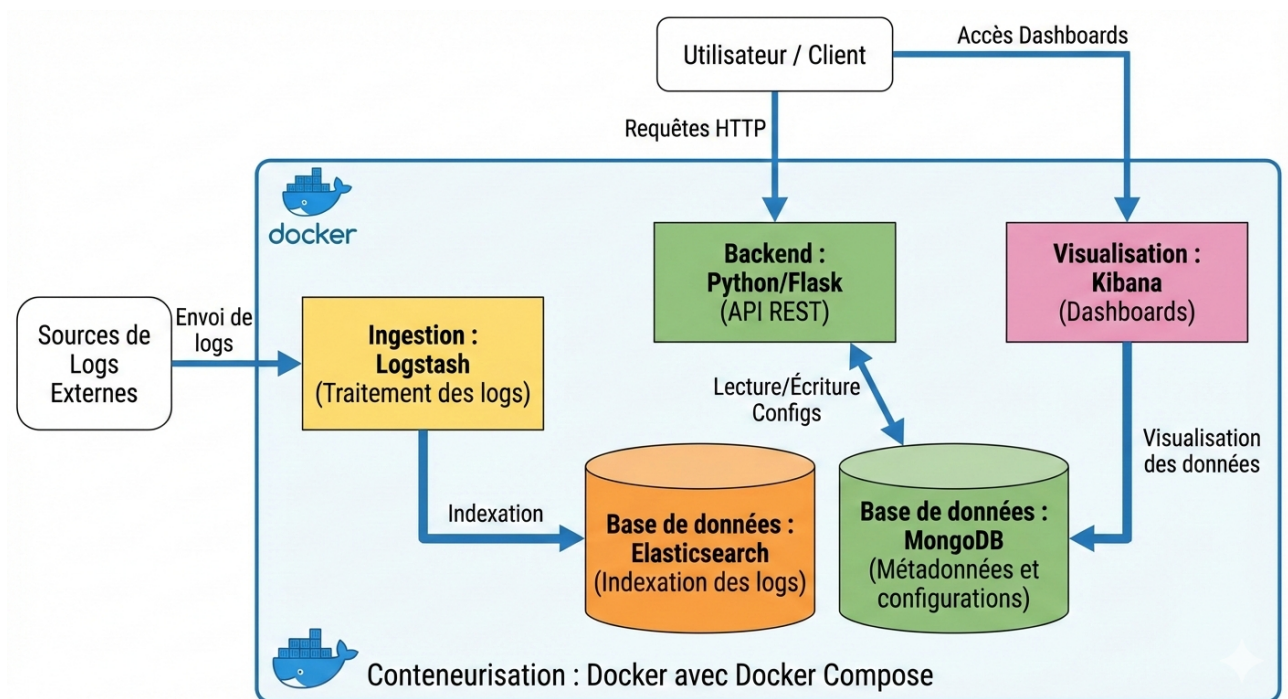


FIGURE 3.1 – Architecture globale de la plateforme

Les logs générés par les applications sont collectés par Logstash, transformés puis envoyés vers Elasticsearch pour indexation. L'API Flask permet d'interagir avec les données, tandis que Kibana assure la visualisation et l'analyse des logs en temps réel.

3.2 Environnement de Développement

3.2.1 Environnement Matériel

Le développement et les tests de l'application ont été réalisés sur un ordinateur portable de marque ASUS disposant des caractéristiques suivantes :

- Processeur : Intel Core i5-8500H @ 2.30 GHz
- Mémoire vive : 16 Go
- Carte graphique : Nvidia GTX 1650
- Stockage : 512 Go SSD
- Système d'exploitation : Windows 64 bits (architecture x64)

Cette configuration matérielle offre des performances suffisantes pour l'exécution simultanée des conteneurs Docker et des outils de monitoring.

3.2.2 Configuration Logicielle

Le tableau suivant présente les principaux outils logiciels utilisés ainsi que leurs versions :

Logiciel	Version
Docker	20.10+
Docker Compose	2.12+
Python	3.9+
Elasticsearch	8.5+
Kibana	8.5+
MongoDB	6.0+
Logstash	8.5+

TABLE 3.1 – Configuration logicielle

3.3 Implémentation des Composants

3.3.1 Docker Compose Principal

L'orchestration des services est assurée par un fichier `docker-compose.yml` qui définit les conteneurs suivants :

- **API Flask** : Fournit les endpoints REST pour la gestion et la recherche des logs.
- **MongoDB** : Stockage persistant des données applicatives.
- **Elasticsearch** : Indexation et recherche des logs.
- **Kibana** : Visualisation et analyse des données.
- **Logstash** : Pipeline d'ingestion et de traitement des logs.

Cette approche permet un déploiement rapide et reproductible de l'ensemble de la plateforme.

3.3.2 Application Flask

L'application Flask constitue le cœur logique de la plateforme. Elle expose des API REST permettant :

- L'ingestion manuelle ou automatique des logs
- La recherche filtrée par niveau, date ou mot-clé
- L'analyse statistique des événements de sécurité

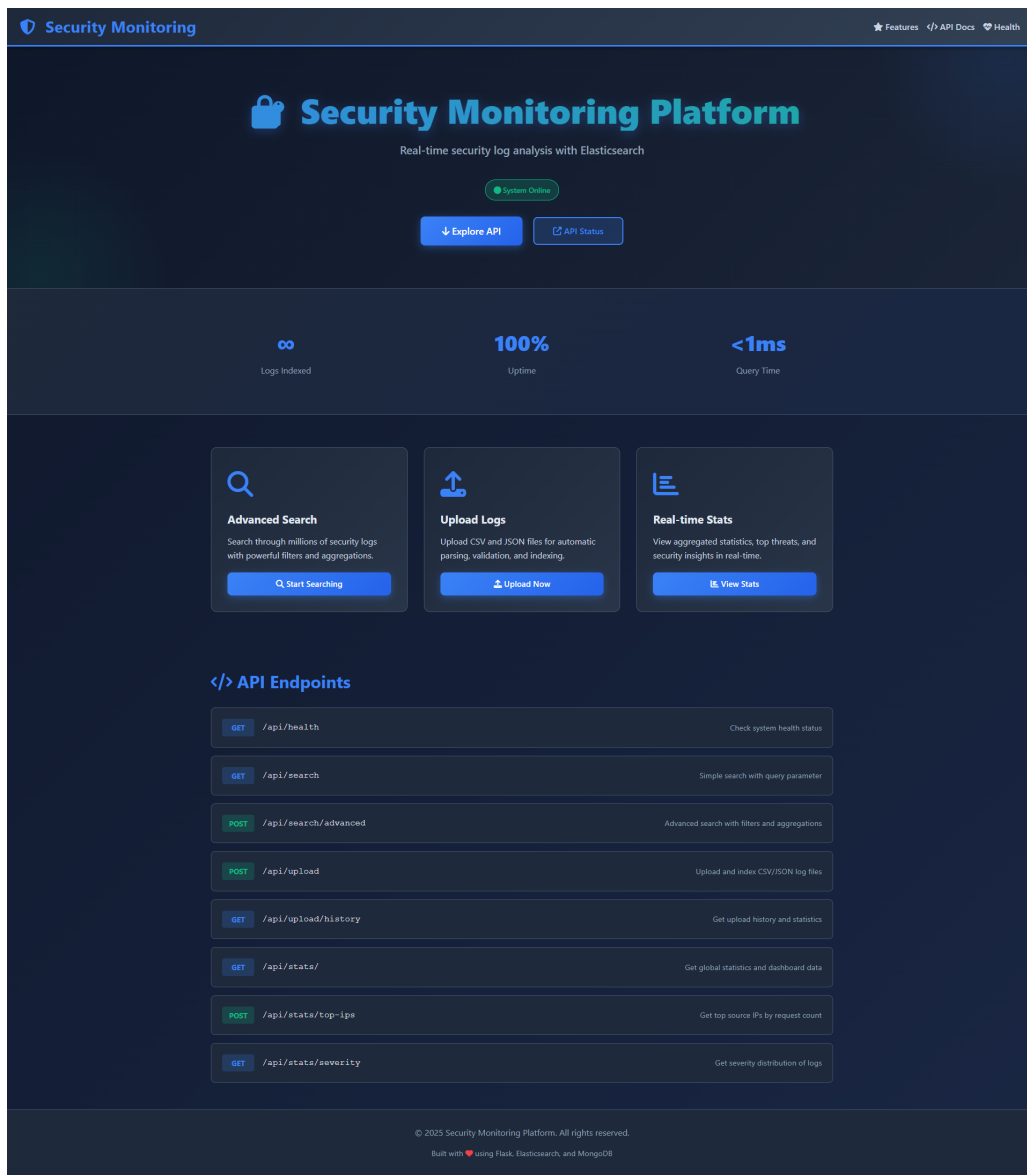


FIGURE 3.2 – Interface principale de l'application Flask

3.3.3 Pipeline Logstash

Le pipeline Logstash est configuré pour :

- Collecter les logs depuis différentes sources
- Appliquer des filtres (par niveau de gravité, timestamp, IP, etc.)
- Normaliser les données avant leur indexation dans Elasticsearch

Cette étape est essentielle pour garantir la cohérence et la qualité des données analysées.

3.3.4 Génération de Logs de Test

Afin de tester la robustesse de la plateforme, des logs de test simulant des événements de sécurité (accès non autorisés, erreurs critiques, tentatives d'attaque) ont été générés automatiquement et injectés dans le système.

3.4 Configuration et Déploiement

3.4.1 Démarrage de la Plateforme

Le déploiement de la plateforme s'effectue à l'aide des commandes suivantes :

```
# Cloner le projet
git clone <repository>
cd security-monitoring-platform

# Démarrer les services
docker-compose up -d

# Vérifier l'état des conteneurs
docker-compose ps

# Arrêter les services
docker-compose down
```

3.4.2 Accès aux Services

Une fois la plateforme démarrée, les services sont accessibles via les URLs suivantes :

Service	URL	Port
Application Web	http ://localhost :8001	8001
Kibana	http ://localhost :5601	5601
Elasticsearch	http ://localhost :9200	9200

TABLE 3.2 – Accès aux services

Conclusion

Ce chapitre a permis de détailler l'architecture technique de la plateforme de monitoring et d'analyse des logs de sécurité mise en place. Nous avons également décrit l'environnement matériel et logiciel utilisé, l'implémentation des principaux composants.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente d’abord les principes de conception adoptés pour assurer une visualisation claire et efficace des événements de sécurité. Ensuite, il détaille l’implémentation des différentes visualisations dans Kibana, les tests d’intégration effectués, ainsi que les optimisations appliquées pour améliorer les performances et l’expérience utilisateur.

4.2 Conception des Tableaux de Bord de Sécurité

La conception des tableaux de bord de sécurité repose sur des principes reconnus dans les centres opérationnels de sécurité (SOC), visant à faciliter la prise de décision rapide et l’investigation des incidents.

1. **Hiérarchisation de l’information** : Les événements critiques et alertes de sécurité sont mis en avant afin d’attirer immédiatement l’attention des analystes.
2. **Temps réel vs analyse historique** : Une distinction claire est établie entre le monitoring en temps réel et l’analyse rétrospective des incidents.
3. **Interactivité** : Les tableaux de bord offrent des capacités de filtrage et d’exploration dynamique des données.
4. **Responsivité** : L’interface est adaptée aux différents écrans utilisés dans un SOC.

Ces principes garantissent une visualisation claire, synthétique et exploitable des données de sécurité.

4.3 Dashboard d’Analyse Avancée

Un tableau de bord dédié à l’investigation approfondie des incidents a été mis en place. Il permet notamment :

- la recherche plein texte avec autocomplétion,
- le filtrage par type d’événement, par pays, adresse IP, etc ..

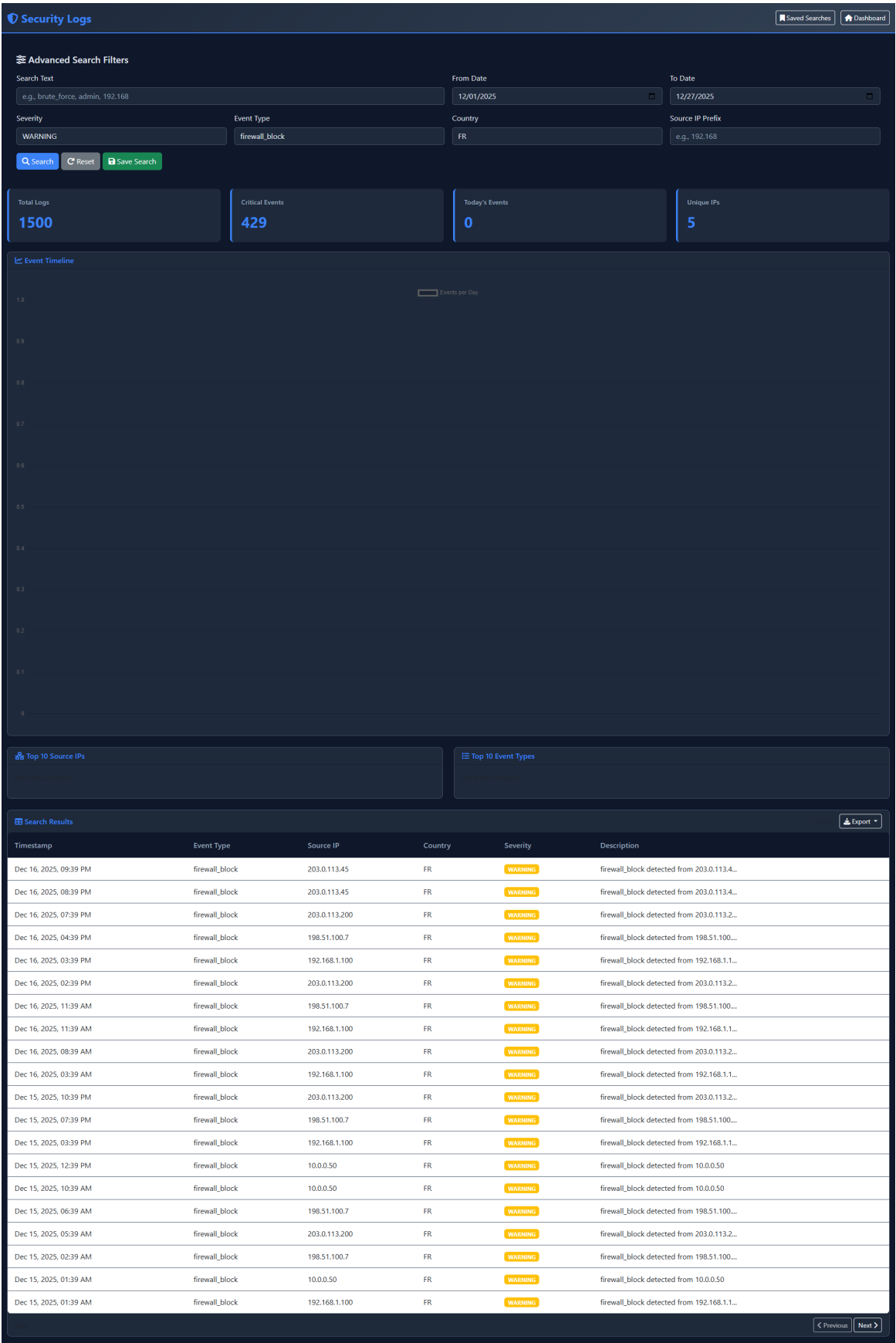


FIGURE 4.1 – Interface de recherche avancée avec filtres

4.4 Implémentation des Visualisations dans Kibana

4.4.1 Configuration des Index Patterns

La configuration des index patterns constitue une étape essentielle pour permettre à Kibana d'accéder aux données stockées dans Elasticsearch et d'optimiser les performances des requêtes.

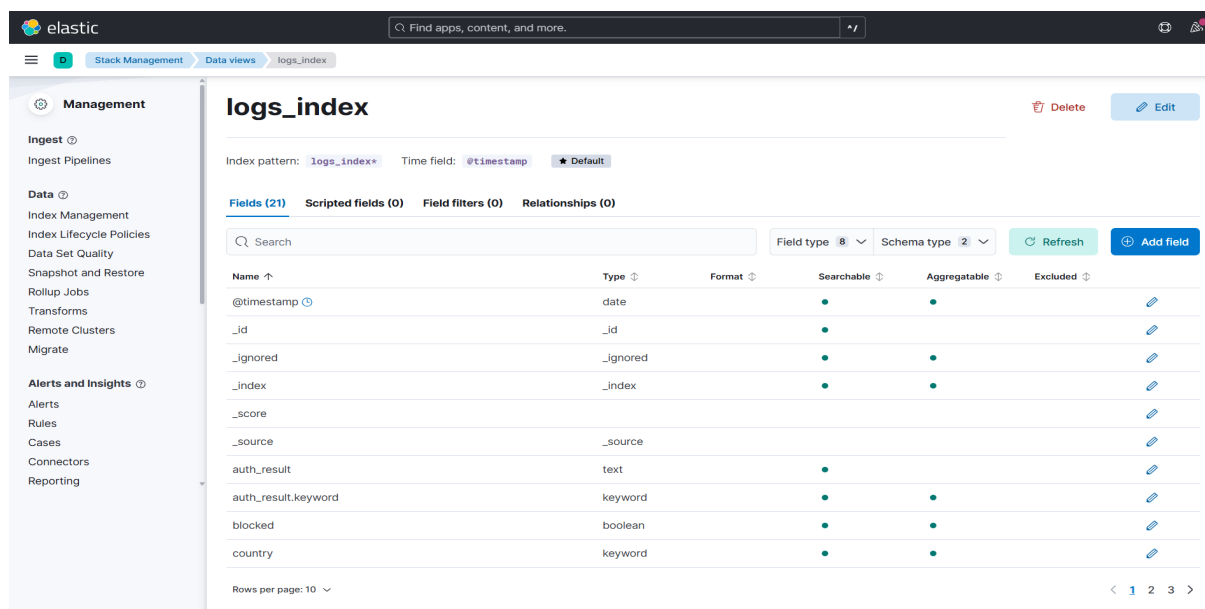


FIGURE 4.2 – Création de l'Index Pattern dans Kibana

4.4.2 Création des Visualisations

Plusieurs types de visualisations ont été développés afin de représenter les données.

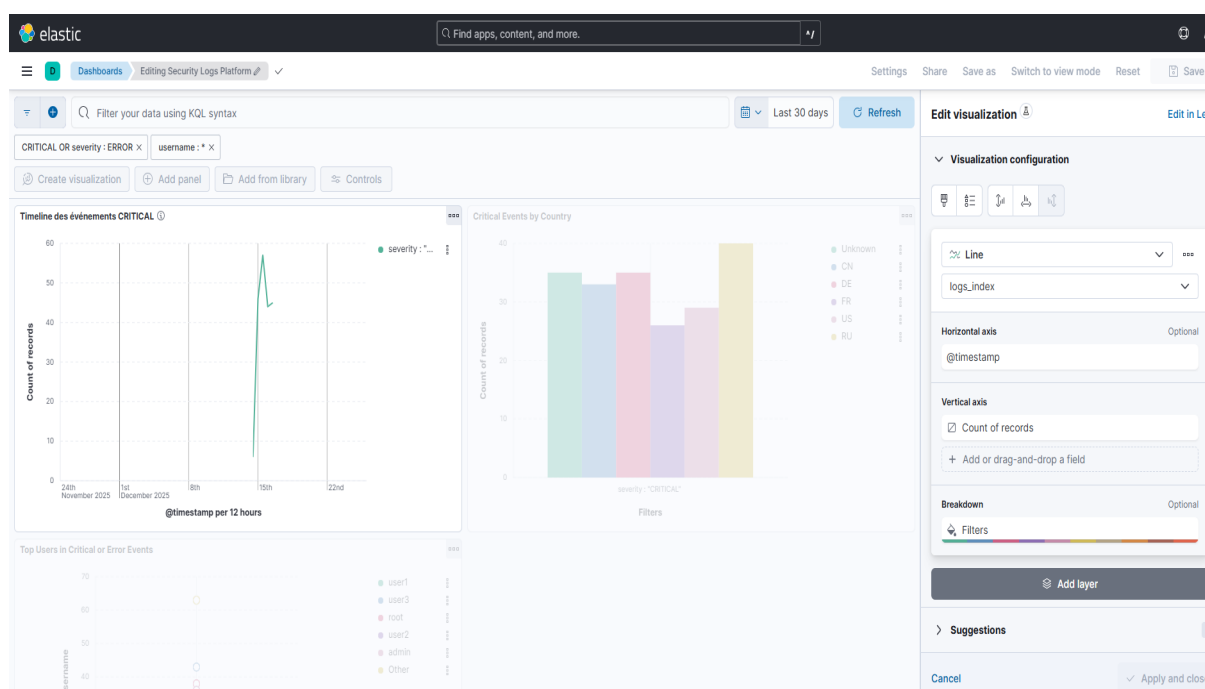


FIGURE 4.3 – Création des visualisations Kibana

4.4.2.1 Création du Dashboard Principal

Le tableau de bord principal fournit une vue synthétique et immédiate de l'état de sécurité du système d'information.

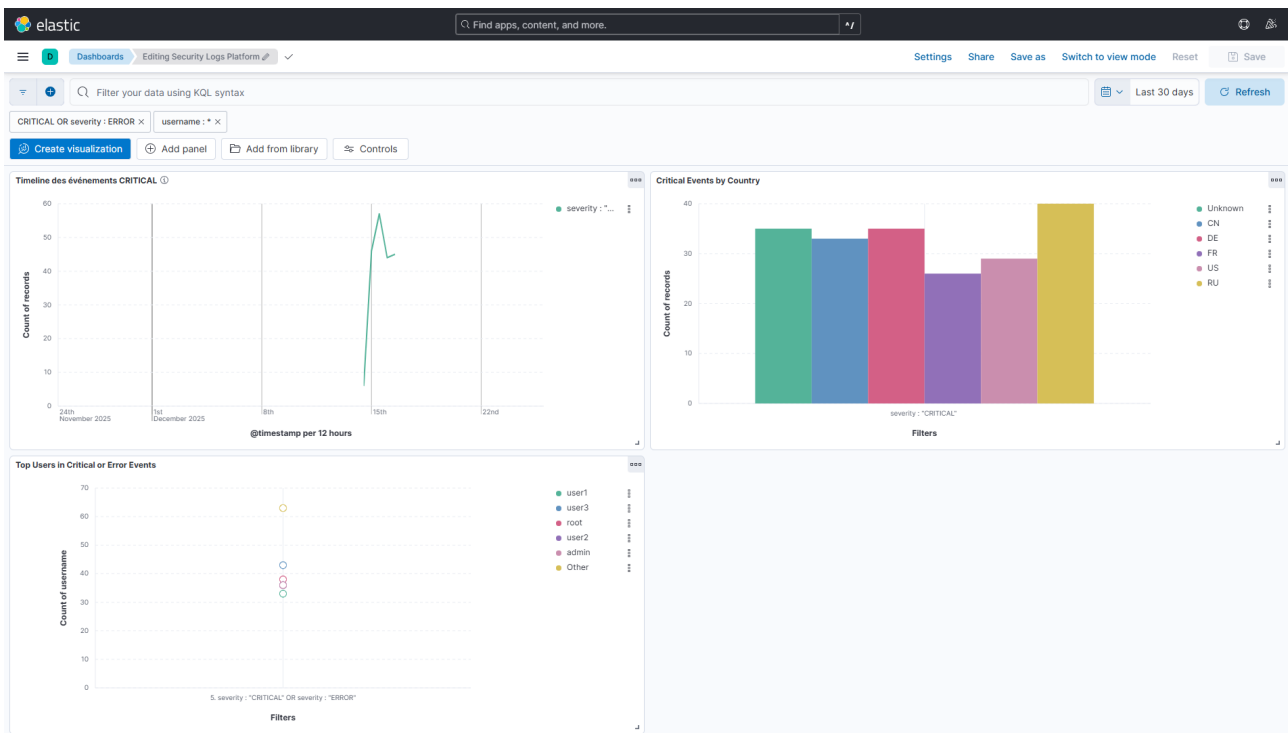


FIGURE 4.4 – Tableau de bord principal de monitoring de sécurité

Il intègre notamment :

- une visualisation de l'évolution temporelle des incidents critiques,
- une carte géographique des pays sources des attaques,
- une identification des comptes utilisateurs à risque.

4.5 Intégration et Tests

4.5.1 Tableaux de Bord Kibana

Plusieurs tableaux de bord ont été conçus dans Kibana afin de faciliter l'analyse visuelle des événements de sécurité :

- Top 10 des adresses IP suspectes,
- géolocalisation des attaques,
- évolution temporelle des événements critiques.

4.5.2 Tests de l'API avec Postman

Les endpoints de l'API ont été testés à l'aide de l'outil Postman afin de valider l'intégration entre la couche applicative et la plateforme Big Data. Les tests ont permis de vérifier :

- le bon fonctionnement de l'ingestion des logs,
- la cohérence et la validité des réponses retournées,
- les performances des requêtes de recherche.

4.5.3 Utilisation de Kibana Canvas

Kibana Canvas a été utilisé pour concevoir des rapports visuels avancés, combinant textes, graphiques et indicateurs clés.

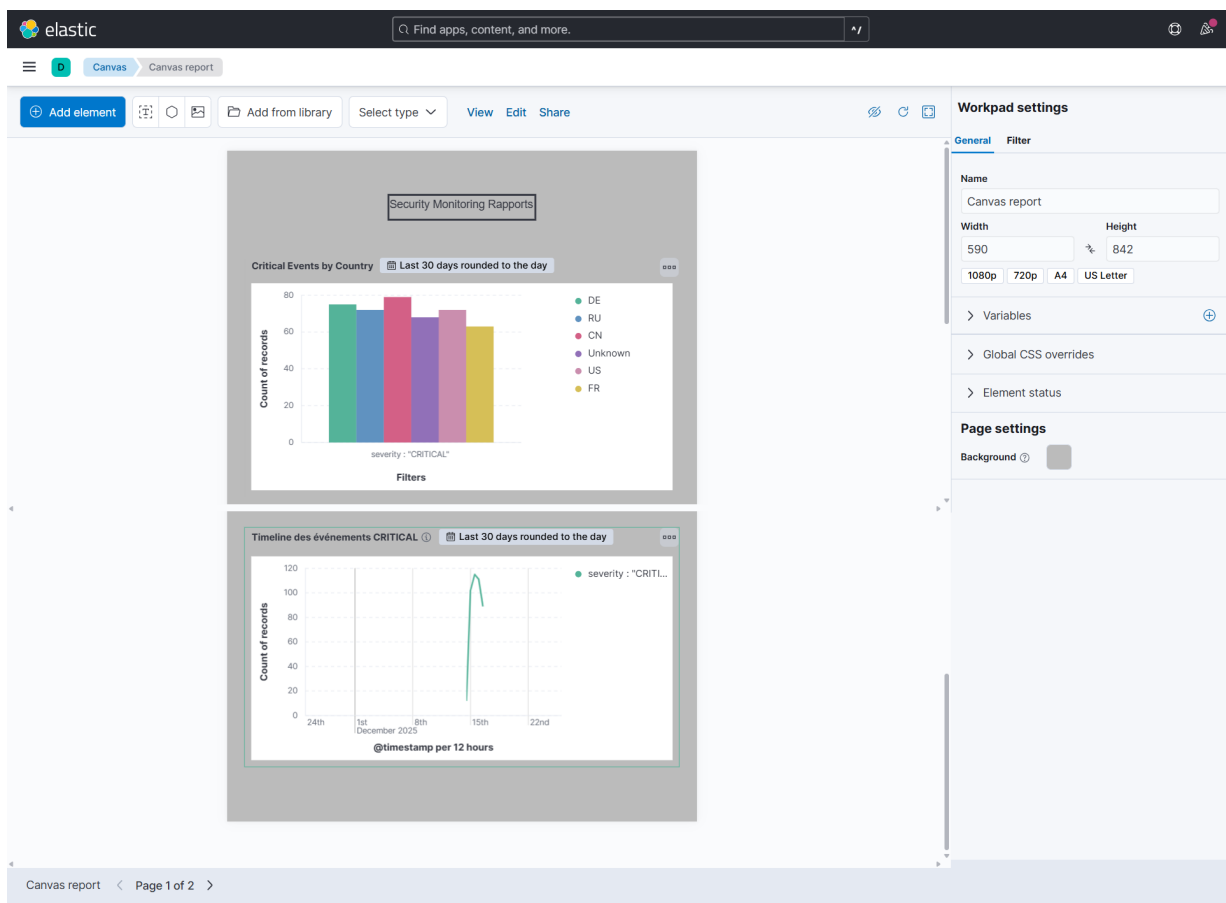


FIGURE 4.5 – Rapport Canvas avec visualisations intégrées

4.6 Tests d'Intégration avec Postman

4.6.1 Configuration des Requêtes API

Les tests Postman ont permis de valider l'intégration complète de l'API avec Elasticsearch et Kibana.

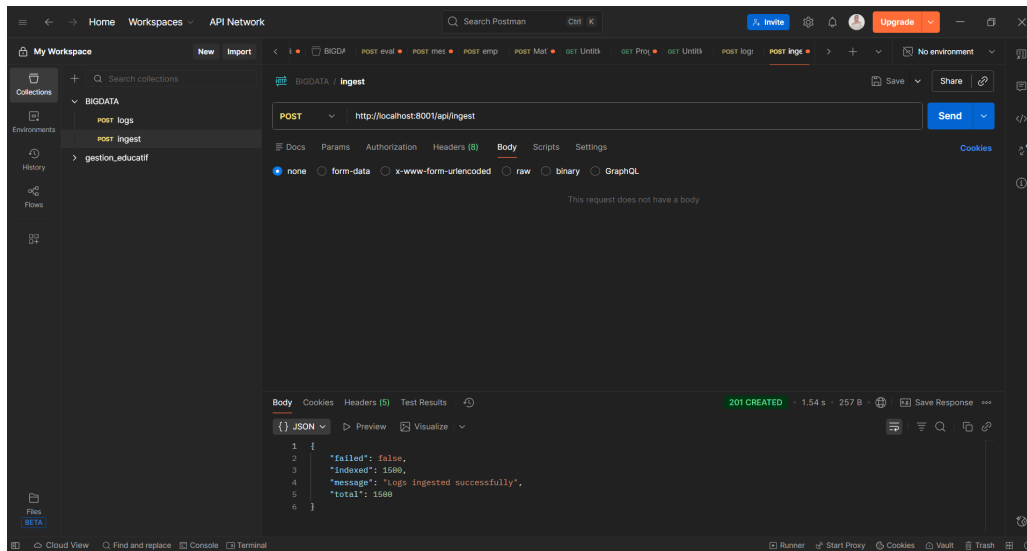


FIGURE 4.6 – Test d'ingestion de logs via Postman

4.7 Optimisations et Bonnes Pratiques

4.7.1 Optimisation des Performances

Plusieurs bonnes pratiques ont été appliquées afin d'optimiser les performances de la plateforme :

- **Indexation** : utilisation de templates d'index et de mappings adaptés,
- **Requêtes** : optimisation des agrégations Elasticsearch,
- **Cache** : mise en cache des visualisations fréquemment consultées,
- **Chargement différé** : chargement progressif des données historiques.

4.8 Conclusion

Ce chapitre a permis de présenter la conception et l'implémentation des tableaux de bord et des visualisations de sécurité au sein de la plateforme de monitoring.

Conclusion générale

En conclusion, ce projet a constitué une expérience enrichissante dans le domaine du Big Data, en particulier à travers la conception et la mise en œuvre d'une plateforme de monitoring et d'analyse des logs de sécurité. Il nous a permis d'aborder de manière concrète les problématiques liées à la gestion de volumes importants de données hétérogènes, générées en continu par les systèmes d'information modernes.

La première phase du travail a consisté à analyser le contexte et à identifier les limites des méthodes traditionnelles de gestion des logs, notamment la dispersion des données, la difficulté d'analyse manuelle et le manque de visibilité globale sur les événements de sécurité. Cette étude nous a conduits à proposer une solution basée sur un framework Big Data, capable de centraliser, traiter et analyser efficacement les logs de sécurité.

À travers les différents chapitres, nous avons étudié les fondements du Big Data et leur importance dans le domaine de la sécurité des systèmes d'information. La stack ELK (Elasticsearch, Logstash et Kibana) a été adoptée comme solution centrale, en raison de ses performances en matière d'indexation, de recherche rapide et de visualisation des données. L'architecture technique mise en place, combinée à une application backend développée avec Flask et à une infrastructure conteneurisée avec Docker, a permis de concevoir une plateforme modulaire, évolutive et facilement déployable.

La phase de réalisation a porté sur l'implémentation concrète de la solution, incluant la configuration des pipelines d'ingestion des logs, la structuration des index Elasticsearch, la création de tableaux de bord interactifs dans Kibana ainsi que la mise en place de fonctionnalités de recherche et d'analyse avancées. Des tests d'intégration ont été réalisés afin de valider le bon fonctionnement de l'API et la fiabilité de l'ensemble du système. Des optimisations ont également été appliquées pour améliorer les performances et la réactivité de la plateforme.

Au-delà des aspects techniques, ce projet a permis de renforcer des compétences transversales essentielles telles que l'analyse des besoins, la conception d'architectures logicielles, la manipulation d'outils Big Data, la conteneurisation des services et la gestion d'un projet technique de bout en bout. Certaines difficultés, notamment liées à la configuration des outils et à l'optimisation des performances, ont été surmontées grâce à une démarche méthodique et expérimentale.

En perspective, plusieurs axes d'amélioration peuvent être envisagés, tels que l'intégration de mécanismes avancés de détection d'anomalies basés sur le machine learning, l'ajout de fonctionnalités d'alerte en temps réel, ou encore le déploiement de la plateforme dans un environnement cloud pour renforcer sa scalabilité et sa résilience.

En somme, ce projet constitue une étape importante dans notre parcours académique et professionnel, en nous offrant une vision concrète de l'apport des technologies Big Data dans le domaine de la sécurité informatique.

Webographie

- Elastic. *Elastic Stack Documentation*.
Disponible sur : <https://www.elastic.co/guide/index.html> [Consulté le 10/12/2025].
- Elastic. *Elasticsearch Reference*.
Disponible sur : <https://www.elastic.co/elasticsearch/> [Consulté le 16/12/2025].
- Elastic. *Kibana User Guide*.
Disponible sur : <https://www.elastic.co/guide/kibana/current/index.html> [Consulté le 16/12/2025].
- Elastic. *Logstash Reference*.
Disponible sur : <https://www.elastic.co/guide/logstash/current/index.html> [Consulté le 16/12/2025].
- Docker. *Docker Documentation*.
Disponible sur : <https://docs.docker.com/> [Consulté le 16/12/2025].
- MongoDB. *MongoDB Documentation*.
Disponible sur : <https://docs.mongodb.com/> [Consulté le 10/12/2025].
- Flask. *Flask Documentation*.
Disponible sur : <https://flask.palletsprojects.com/en/2.3.x/> [Consulté le 10/12/2025].